

1. processid

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("My Process ID (PID) = %d\n", getpid());

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo x + v
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ nano processid.c
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc processid.c -o processid
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./processid
My Process ID (PID) = 721
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```

2. creating child using fork()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ nano fork.c
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc fork.c -o fork
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./fork
Parent Process
PID = 767
Child Process
PID = 768
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```

3. Parent Waits for Child (wait())

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process Executing...\n");
        sleep(3);
        printf("Child Finished\n");
    }
    else
    {
        wait(NULL);
        printf("Parent Process Continues\n");
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ nano wait.c
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc wait.c -o wait
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./wait
Child Process Executing...
Child Finished
Parent Process Continues
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```

4. Multiple Child Processes

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    for(int i=1;i<=3;i++)
    {
        if(fork()==0)
        {
            printf("Child %d PID = %d\n", i, getpid());
            return 0;
        }
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo x + v
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ nano multiple_children.c
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc multiple_children.c -o multiple_children
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./multiple_children
Child 1 PID = 899
Child 2 PID = 900
Child 3 PID = 901
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```

5. Execute linux command using execlp()

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    printf("Before exec()\n");

    execlp("ls", "ls", "-l", NULL);

    printf("This will not execute.\n");

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ nano execlp.c
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ gcc execlp.c -o execlp
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ ./execlp
Before exec()
total 100
-rwxr-xr-x 1 rehan_lenovo rehan_lenovo 16000 Aug 12 06:45 execlp
-rw-r--r-- 1 rehan_lenovo rehan_lenovo 179 Aug 12 06:45 execlp.c
-rwxr-xr-x 1 rehan_lenovo rehan_lenovo 16080 Aug 12 06:40 fork
-rw-r--r-- 1 rehan_lenovo rehan_lenovo 296 Aug 12 06:40 fork.c
-rwxr-xr-x 1 rehan_lenovo rehan_lenovo 16048 Aug 12 06:44 multiple_children
-rw-r--r-- 1 rehan_lenovo rehan_lenovo 228 Aug 12 06:43 multiple_children.c
-rwxr-xr-x 1 rehan_lenovo rehan_lenovo 16000 Aug 12 06:39 processid
-rw-r--r-- 1 rehan_lenovo rehan_lenovo 122 Aug 12 06:37 processid.c
-rwxr-xr-x 1 rehan_lenovo rehan_lenovo 16072 Aug 12 06:43 wait
-rw-r--r-- 1 rehan_lenovo rehan_lenovo 335 Aug 12 06:43 wait.c
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ |
```

6. Parent and Child Executing Concurrently

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    fork();

    for(int i=1;i<=5;i++)
    {
        printf("PID: %d Count: %d\n", getpid(), i);
        sleep(1);
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ nano concurrent_processes.c
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ gcc concurrent_processes.c -o concurrent_processes
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ ./concurrent_processes
PID: 1004 Count: 1
PID: 1005 Count: 1
PID: 1004 Count: 2
PID: 1005 Count: 2
PID: 1004 Count: 3
PID: 1005 Count: 3
PID: 1004 Count: 4
PID: 1005 Count: 4
PID: 1004 Count: 5
PID: 1005 Count: 5
rehan_lenovo@Rehan-Lenovo: ~/practical-3-4$ |
```

7. Zombie Process

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Exiting\n");
        return 0;
    }
    else
    {
        sleep(15);
        printf("Parent Finished\n");
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo  ×  +  v
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc zombie_process.c -o zombie_process
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./zombie_process
Child Exiting
Parent Finished
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```

8. OrphanProcess

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if(pid > 0)
    {
        printf("Parent Exiting\n");
        return 0;
    }
    else
    {
        sleep(5);
        printf("Child PID : %d\n", getpid());
        printf("New Parent PID : %d\n", getppid());
    }

    return 0;
}
```

Output:

```
rehan_lenovo@Rehan-Lenovo x + v
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ nano OrphanProcess.c
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ gcc OrphanProcess.c -o OrphanProcess
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ ./OrphanProcess
Parent Exiting
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ Child PID : 1140
New Parent PID : 386
^C
rehan_lenovo@Rehan-Lenovo:~/practical-3-4$ |
```